ARMY INSTITUTE OF TECHNOLOGY, PUNE

DEPARTMENT OF COMPUTER ENGINEERING



COMPUTER GRAPHICS PROJECT DOCUMENTATION REPORT

CHRONO-DASH: ANTIGRAVITY ARENA

A 3D First-Person Sci-Fi Shooter with Antigravity Mechanics built with C++ and OpenGL 3.3 Core

SUBMITTED BY

Suryansh Chauhan Roll No. 7240	**Computer Engineering** BE / TE Computer Graphics

Computer Graphics & Game Development

PROJECT DOCUMENTATION

Contents & Report Roadmap

PAGE	SECTION	WHAT IT COVERS
3	1. Executive Summary & Project Snapshot	Project purpose, premise, deliverable facts and core screenshot
4	2. Introduction, Need & Objectives	Problem statement, graphics learning goals and outcomes
5	3. Concept, Theme & Gameplay Loop	Player journey, zero-G arena combat, scoring and state flow
6	4. Technology Stack & Project Organisation	C++17, OpenGL 3.3 Core, GLFW, GLAD, GLM, build pipeline
7	5. System Architecture & Game Flow	Runtime ownership, simulation integration, and state transitions
8	6. Player Mechanics, Input & Camera	FPS camera freelook, kinematic movement, jump and thrusters
9	7. Arena Design & Procedural World Construction	Modular platforms, anchor cubes, procedural batching
10	8. Antigravity Core, Hazards, Enemies & Weapons	Plasma orb, drone steering AI, pulse lasers, void hazards
11	9. Transformations & Projection	Affine 4x4 matrix chains, orthonormal look-at, perspective projection
12	10. Lighting, Shading & Materials	Blinn-Phong illumination, point attenuation, rim light, materials
13	11. Hierarchical Animation & Orientation Blending	Viewmodel walk-bobbing, mouse sway lag, and recoil kickback
14	12. Depth, Fog & Rendering Passes	24-bit Z-buffer, atmospheric distance fog, 3-pass compositing
15	13. Physics, Collision & Gravity Mechanics	AABB platform collision, sphere overlap, Euler integration
16	14. Scoring, Progression & Level Walkthrough	Multiplier formulas, consecutive streak timers, combat flow
17	15. Interface & HUD	Vector stroke typography, health/ammo meters, reticle overlay
18	16. Testing, Validation & Limitations	Automated compilation verification, checklist and known limits
19	17. Conclusion, Future Scope & References	Summary, future graphics expansions, and citations

1. Executive Summary & Project Snapshot

Chrono-Dash uses a dynamic zero-G anomaly at the center of a floating cyberpunk arena to make 3D graphics concepts interactive and observable.

Chrono-Dash: Antigravity Arena is a high-speed 3D first-person shooter (FPS) implemented in modern C++17 and the OpenGL 3.3 Core Profile. The player maneuvers across floating metallic platforms suspended over a twilight sci-fi void, confronting autonomous hostile drones. The defining mechanical centerpiece is a massive, pulsing Antigravity Core suspended in the center of the arena. Entering its 16.5-meter radius flips downward terrestrial gravity into near-zero weightlessness, allowing full 3D vertical movement where players jump, thrust, and dodge as if in deep space.

The distinguishing engineering feature is that the game is built from first principles as an interactive computer graphics demonstration rather than relying on a commercial black-box engine. The graphics pipeline features programmable GLSL 330 vertex and fragment shaders executing a per-fragment Blinn-Phong lighting model with dynamic point-light distance attenuation, a procedural wireframe floor grid, a glowing plasma sphere, and distance fog. All 3D meshes (cubes, spheres, octahedrons) and 2D vector HUD characters are generated procedurally in code without external texture or font files.

1.1 Project facts

ITEM	DETAIL
Project name	CHRONO-DASH: Antigravity Arena - 3D Sci-Fi FPS & Zero-G Platformer
Platform / toolchain	Windows 10/11; MinGW-w64 (GCC 16.1.0), CMake 3.16+, Ninja, Release x86_64
Graphics API	Modern OpenGL 3.3 Core Profile; GLSL 3.30 vertex and fragment shaders
Libraries	GLFW 3.3 (window, input, 60 FPS VSync), GLAD 2 (loader), GLM (mathematics)
Primary language	C++17 for game simulation, math & physics; GLSL for shading (~2,100 lines)
Core objective	Survive hostile drone waves, engage zero-G dogfights, build 5x kill streaks
Scoring	100 pts per kill x Streak Multiplier (1x, 2x, 3x, 5x) with 4.5s countdown gauge
Local project folder	CgGameCie3/ (include/, src/, build.bat, run.bat, CMakeLists.txt)



Figure 1.1 In-game capture: Floating arena, Antigravity Core orb, enemy drone combat, and 2D vector HUD telemetry.

2. Introduction, Need & Objectives

The primary goal is to synthesize core computer graphics theory into a responsive, real-time interactive simulation.

2.1 Background and problem statement

Computer Graphics unifies linear algebra, projective geometry, optical illumination physics, and real-time rasterization. In traditional academic curricula, these topics are often taught as disjoint equations: students learn matrix multiplication, then write a static shader, and separately study camera projection. Consequently, learners struggle to comprehend how coordinate spaces, viewing frustums, dynamic lights, and physical collision bounds cooperate synchronously inside a single real-time program.

Chrono-Dash bridges this pedagogical gap through a gameplay mechanic that continually exercises coordinate systems. Entering the central core dynamically alters acceleration vectors, switches the camera viewing physics, activates 3D vertical thrusters, and recalculates weapon sway matrices. Every gameplay element represents an applied computer graphics principle that the player can see and manipulate in real time.

2.2 Objectives

• Engineer a high-performance 3D FPS game loop in modern C++17 running locked at 60 FPS via GLFW and GLAD.

• Explicitly implement the 3D graphics pipeline: compose Model, View, and Perspective Projection matrices on the CPU via GLM and upload them as shader uniforms.

• Write programmable GLSL 330 vertex and fragment shaders implementing per-fragment Blinn-Phong shading, point-light attenuation, Fresnel rim lighting, and atmospheric distance fog.

• Generate all 3D geometries procedurally in C++ (cubes, UV spheres, octahedrons) with analytical vertex normals.

• Implement a 3-pass rendering architecture that isolates the 3D world, viewmodel weapon (with depth buffer clearing), and an orthographic 2D vector HUD overlay.

• Build collision detection from first principles using Axis-Aligned Bounding Box (AABB) tests and Sphere-Sphere overlap tests without external physics libraries.

2.3 Relevance to the assignment

The project directly satisfies the practical Computer Graphics syllabus requirements. Every game component serves a dual role: gameplay engages the user, while the underlying architecture demonstrates fundamental graphics concepts (affine transformations, viewing pipelines, rasterization, and illumination).

2.4 Expected learning outcomes

LEARNING OUTCOME	EVIDENCE A STUDENT CAN EXPLAIN
Mathematical Transformations	Why matrix multiplication order P x V x M matters, and how local coordinate frames transform into world and camera spaces.
Lighting & Illumination	How ambient, diffuse, and specular terms combine, why Blinn-Phong uses a halfway vector H, and how distance attenuation works.
Pipeline & Depth Control	How depth buffer clearing (glClear(GL_DEPTH_BUFFER_BIT)) prevents weapon-wall clipping in first-person rendering.
Procedural Shading	How screen-space partial derivatives (fwidth) render infinite anti-aliased grid lines without textures.

3. Concept, Theme & Gameplay Loop

A floating cyberpunk arena, an antigravity energy core, and hostile drone swarms synthesize graphics and combat mechanics.

3.1 Game premise

The player assumes the role of a chrononaut fighter stranded within a floating training arena suspended in a deep-space twilight void. The arena is composed of geometric metallic platforms hovering at varying heights around a massive central gravitational distortion anomaly. Hostile drone interceptors spawn periodically around the perimeter, closing distance and firing ballistic purple laser bursts. The player must utilize both grounded sprint-and-jump maneuvers and 3D zero-G aerial thrusting to out-position adversaries, fire cyan laser bolts, and build rapid kill streaks.

3.2 Core gameplay loop

1. Spawn on the main central platform (0.0, 2.5, 6.0) with standard 1.0G downward gravity.

2. Navigate with W/A/S/D and aim with smooth mouse freelook camera control.

3. Enter the central Antigravity Core radius (R = 16.5m) to activate zero-G floatiness and engage vertical 3D thrusters (Space to ascend, Shift/Ctrl to descend).

4. Track hostile octahedron drones, leading targets and firing high-velocity cyan laser projectiles (Left Mouse Button).

5. Eliminate targets in rapid succession to maintain consecutive hit streaks, boosting the score multiplier up to 5x before the 4.5-second timer expires.

6. Manage ammo consumption (30-round magazine, 90 reserve) and initiate tactical reloads with R.

3.3 Arena layout overview and combat zones

ARENA ZONE	ELEVATION & EXTENTS	GRAVITATIONAL STATE
Main Central Hub	y = 0.0m; 26m x 1.2m x 26m	Ground physics (g = -25.0 m/s²) / Core boundary
North Elevated Platform	y = 6.0m; 16m x 1.0m x 12m	Terrestrial gravity (Sniper / tactical vantage)
South Elevated Platform	y = 7.5m; 16m x 1.0m x 12m	Terrestrial gravity (Secondary cover platform)
East & West High Perches	y = 11.5m; 12m x 1.0m x 14m	Near zero-G boundary / Drone intercept vectors
Antigravity Distortion Sphere	y = 10.5m; Radius = 16.5m	Zero-G Active: g = -2.2 m/s², Space/Shift 3D thrusters

Scoring Formula: Score = Σ [Base Kill (100 pts) x Multiplier (1x, 2x, 3x, 5x)]. A streak timer of 4.5 seconds resets with each confirmed hit. Eliminating 2, 5, or 9 targets within the active streak window advances the multiplier to 2x, 3x, and 5x respectively.

4. Technology Stack & Project Organisation

A clean C++17 architectural structure paired with hardware OpenGL 3.3 Core specifications and lightweight dependencies.

LAYER	TECHNOLOGY / COMPONENT	ROLE IN CHRONO-DASH
Language	C++17 (MSVC / MinGW GCC)	Game simulation loop, procedural geometry generation, entity AI, math and physics
Graphics API	Modern OpenGL 3.3 Core Profile	Hardware-accelerated pipeline: VAO/VBO/EBO management, depth tests, multisample, alpha blend
Shaders	GLSL 3.30 (3D Scene & 2D HUD)	Model-View-Projection matrix transforms, Blinn-Phong shading, procedural floor grid, core plasma
Window / Input	GLFW 3.3	Window context creation, 4x MSAA framebuffer hints, VSync timer, raw mouse cursor locking
Loader	GLAD 2 (gl.h)	Dynamically interrogates driver exports at runtime to retrieve modern OpenGL function pointers
Math	GLM (glm/glm.hpp, gtc)	Vector/matrix math (vec2, vec3, mat4, translate, rotate, scale, perspective, ortho, lookAt)
Build System	CMake 3.16+ & Ninja / build.bat	Cross-platform build automation generating optimized standalone executable ChronoDash.exe (~870 KB)
Assets	Procedural in-source generation	All meshes (cubes, spheres, octahedrons) and HUD vector strokes are synthesized in code

4.1 Project structure

• **include/** - Header declarations: OpenGLHeaders.h, Constants.h, Types.h, Shader.h, Mesh.h, UIRenderer.h, Player.h, GameState.h, GameWorld.h, Renderer.h, glad/gl.h, KHR/khrplatform.h.

• **src/** - Implementation source files: main.cpp (application entry and loop), gl.c (loader), Shader.cpp, Mesh.cpp, UIRenderer.cpp, Player.cpp, GameState.cpp, GameWorld.cpp, Renderer.cpp.

• **media/** - Generated documentation graphics, diagrams, and high-fidelity gameplay screenshots.

• **build.bat & run.bat** - One-click Windows build and launch batch scripts supporting both CMake and direct g++ compiling.

• **CMakeLists.txt & README.md** - Cross-platform CMake configuration and comprehensive deployment instructions.

4.2 Custom modular design philosophy

Chrono-Dash was intentionally architected into a modular, decoupled engine structure. Separating rendering logic from game simulation guarantees clean code maintainability: the Renderer class oversees multi-pass GPU drawing, the GameWorld class governs entity lifecycles and collision response, the Player class manages FPS camera math, and the UIRenderer class rasterizes resolution-independent vector typography. All code compiles natively in under 5 seconds with zero external runtime DLL requirements beyond standard system OpenGL libraries.

5. System Architecture & Game Flow

The engine decouples windowing, user input, simulation physics, enemy AI, and multi-pass presentation.

5.1 Runtime organisation

MODULE	RESPONSIBILITY	KEY FILES
Entry & Window	Initializes GLFW window, 4x MSAA context, GLAD loader, input callbacks, delta-time timer	main.cpp
GameWorld	Central simulation: entity arrays, arena platforms, floating props, collision solver, projectile step	GameWorld.h / .cpp
Player	FPS camera state, yaw/pitch accumulation, velocity, weapon recoil/sway/bobbing, ammo/health	Player.h / .cpp
Renderer	Oversees 3-pass graphics pipeline: 3D world pass, viewmodel weapon pass, and 2D HUD pass	Renderer.h / .cpp
UIRenderer	Procedural vector stroke generator converting glyph line segments into screen-space 2D quads	UIRenderer.h / .cpp
Shaders	GLSL 330 core programs: Blinn-Phong shading, point lights, procedural grid, core plasma, 2D HUD	Shader.h / .cpp
Mesh Factory	Procedural VAO/VBO buffer factories for unit cubes, UV spheres, and octahedrons	Mesh.h / .cpp

5.2 Frame and state flow

Each 60 FPS tick, the main loop calculates delta time (dt clamped to max 50ms for numerical physics stability) and polls GLFW events. The GameWorld updates player movement, checks Antigravity Core proximity, resolves platform AABB collisions, advances projectile rays, updates enemy AI tracking/strafing/firing, spawns particles, and evaluates streak countdown timers. The Renderer then clears depth/color buffers, executes Pass 1 (3D world), clears the depth buffer to draw Pass 2 (viewmodel gun), and disables depth testing for Pass 3 (2D vector HUD overlay).

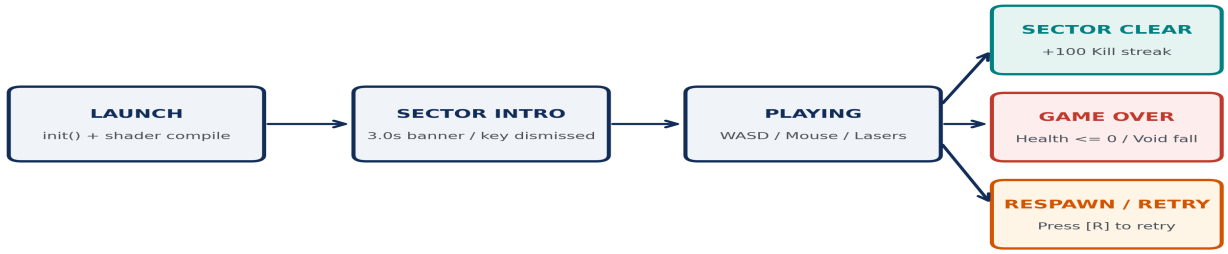


Figure 5.1 Game runtime state machine and transition flow. ESC toggles PAUSE without leaving PLAYING.

5.3 State transitions

CURRENT STATE	TRIGGER EVENT	SIMULATION RESULT
PLAYING	Player health <= 0 or void fall (y < -12.0m)	Switches to GAME OVER; displays red vignette and mission failed modal
GAME OVER	Player presses R	Resets player coordinates, restores HP to 100, resets score and enemies
PLAYING	Player presses R (with ammo missing)	Initiates 1.25s tactical weapon reload sequence
PLAYING	ESC key pressed	Toggles mouse cursor lock between FPS captured mode and desktop free cursor

6. Player Mechanics, Input & Camera

First-person synthetic camera control, kinematic ground response, and 3D zero-G aerial maneuverability.

6.1 Movement and physical response

Player movement combines responsive kinematic ground acceleration with physical inertia damping. On solid platforms, horizontal movement velocity accelerates up to moveSpeed = 13.5 m/s along forward and right camera vectors. Standard terrestrial downward gravity operates at g = -25.0 m/s², while pressing Space imparts an instantaneous vertical impulse jumpSpeed = 9.2 m/s. When entering the Antigravity Core radius (d < 16.5m), downward gravity diminishes to g = -2.2 m/s²: holding Space engages continuous upward thrusters (+14.0 m/s²), while holding Shift or Ctrl engages downward thrusters (-14.0 m/s²). Inertial drag is softened via velocity attenuation factor v *= (0.85)^(dt*60), giving a fluid floating sensation.

6.2 Control reference table

INPUT	ACTION EXECUTED	SYSTEM PURPOSE & MECHANIC
W / S	Move Forward / Backward	Translates player along horizontal forward camera vector
A / D	Strafe Left / Right	Translates player along orthogonal horizontal right camera vector
Mouse Movement	FPS Freelook Camera	Accumulates delta pitch and yaw to rotate synthetic camera view
Spacebar	Jump / Upward Thruster	Standard vertical jump on platforms; continuous upward climb in zero-G
Shift / Ctrl	Downward Thruster	Active downward descent thrust when within the Antigravity Core field
Left Mouse Button	Fire Pulse Cannon	Spawns high-velocity cyan laser bolt (72 m/s) with weapon recoil
R Key	Reload Weapon / Respawn	Reloads 30-round magazine (1.25s timer); respawns player after Game Over
Escape	Toggle Cursor Capture	Releases raw mouse trap to inspect OS desktop / pauses gameplay

6.3 Synthetic first-person camera implementation

The first-person camera builds its view matrix via glm::lookAt(eye, eye + forward, up). The eye point is clamped at player.pos + vec3(0.0, 1.70, 0.0), matching the human eye height. The forward directional vector is derived from spherical coordinates: F = (cos(yaw)*cos(pitch), sin(pitch), sin(yaw)*cos(pitch)), with pitch clamped strictly between -89.0° and +89.0° to eliminate gimbal lock singularity. The right vector is derived via cross product R = normalize(cross(F, vec3(0,1,0))), and up is derived via cross(R, F).

7. Arena Design & Procedural World Construction

Modular geometric floating platforms, anchor cubes, and procedural vertex generation without imported mesh assets.

7.1 Arena construction pipeline

1. Define the arena spatial volume: 60 x 60 meter horizontal area hovering over an infinite-feeling void plane at y = -14.0m.

2. Generate unit cube primitive (24 vertices, 36 indices, 6 face normals) and UV sphere primitive (24 stacks x 28 sectors) in C++.

3. Construct the central main platform (26m x 1.2m x 26m) at origin (0, 0, 0) with a dark metallic body and glowing neon cyan perimeter trims.

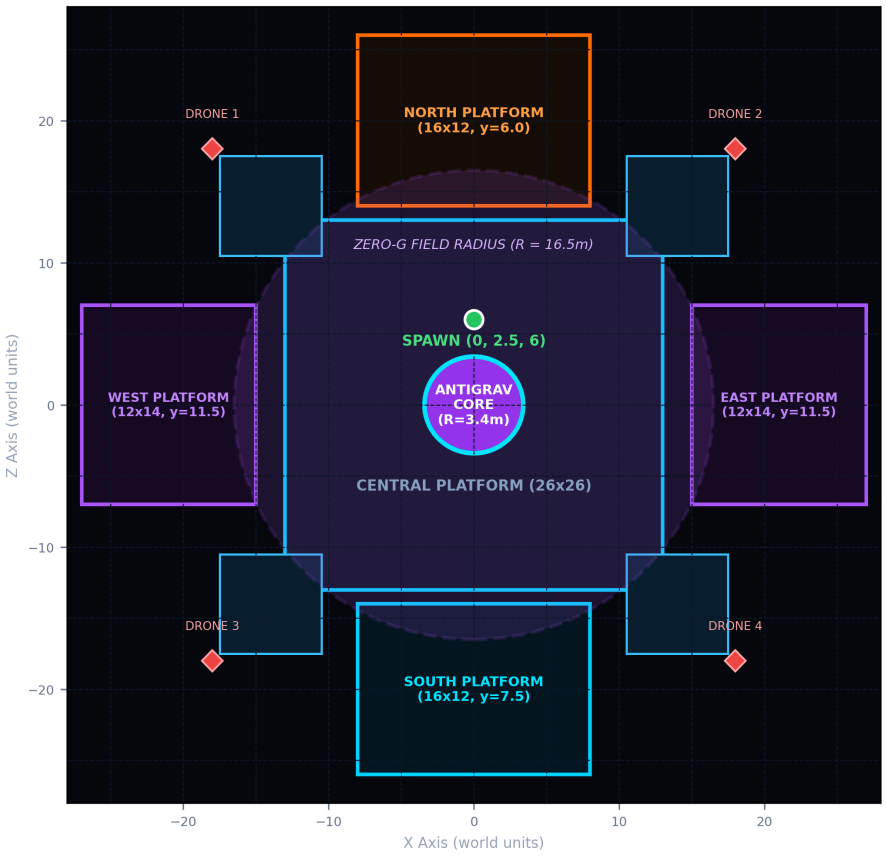
4. Position elevated tactical platforms: North platform (16x12 at y=6.0m), South platform (16x12 at y=7.5m), West/East high perches (12x14 at y=11.5m), and four corner stepping pads (7x7 at y=3.8m).

5. Spawn floating zero-G anchor cubes and 18 drifting debris chunks orbiting the core with procedural rotational axes and floating sinusoidal offsets.

7.2 Technical characteristics table

PROPERTY	DOCUMENTED IMPLEMENTATION
Arena Dimensions	60m x 30m x 60m overall operational airspace; central hub 26m x 26m
Primitive Mesh Types	Unit Cube (36 indices, flat face normals), UV Sphere (672 tris), Octahedron (8 tris)
Material Batches	Matte metallic platform base, neon cyan/orange/purple emissive trims, plasma core
Normals Generation	Explicit outward analytical normals per vertex guaranteeing accurate Blinn-Phong shading
Floor Grid Void	Procedural GLSL shader plane (180m x 180m) evaluating fwidth anti-aliased grid lines at y = -14.0m
Colliders	Analytic Axis-Aligned Bounding Boxes (AABBs) stored in each Platform struct

Figure 7.1 Top-down map of Chrono-Dash Floating Arena
(Cyan: Lower/Ground, Orange: North, Purple: High Platforms, Purple Dashed: Zero-G Field, Red: Drone Spawns)



8. Antigravity Core, Hazards, Enemies & Weapons

Dynamic interactive game entities governed by custom mathematical algorithms and shader simulations.

8.1 Central Antigravity Core

Suspended at coordinates (0.0, 10.5, 0.0), the Antigravity Core is an ethereal plasma sphere (radius 3.4m) surrounded by a rotating gravitational distortion ring (5.0m outer radius). Within the GLSL fragment shader, the core is rendered with an animated swirling plasma pattern: sin(pos.y * 3.5 + t * 4.0) + cos(pos.x * 2.8 + pos.z * 2.8 + t * 3.0), blended with an intense Fresnel rim glow (1 - N·V)^2.2. The core also acts as a dynamic point-light emitter casting purple-blue illumination across the entire arena.

8.2 Pulse Cannon & laser projectiles

The player's weapon fires high-velocity cyan laser bolts (velocity = 72.0 m/s along the crosshair aim ray, lifetime = 2.2s). Each bolt is modeled as an elongated cylindrical cube scaled to (0.12m, 0.12m, 0.95m) with high emissive brilliance (emissive * 3.0). Discharging the weapon triggers an instantaneous muzzle flash point light (intensity = 1.0) and initiates kickback recoil.

8.3 Autonomous hostile drone interceptors

Hostile drones spawn periodically around the perimeter. Each drone is constructed from a faceted metallic octahedron core with a glowing red front optical sensor eye and lateral stabilizer wings with red thruster exhausts. Drones execute an autonomous AI steering loop: they track the player's 3D position, maintain an optimal combat standoff distance between 8m and 18m, execute lateral orbital strafing via cross(dir, up), and bob vertically with sinusoidal frequency sin(t * 2.5 + phase) * 1.8. Every 2.0 to 3.5 seconds, drones fire high-speed purple laser bolts (26.0 m/s) toward the player's predicted position.

8.4 Void boundary & hazard safety repulsion

Falling off platforms into the void below y = -12.0m trips the station's emergency gravitational repulsion field: the player is launched upward at +19.0 m/s, penalized with 12 damage points, and alerted with a red screen damage vignette.

9. CG Concepts I: Transformations & Projection

Every entity is transformed by a sequential chain of 4x4 homogeneous affine matrices uploaded as shader uniforms.

9.1 Homogeneous transformation chains

A 3D coordinate is represented as a homogeneous 4-vector (x, y, z, 1). Matrix multiplication composes transformations from right to left. Every world object, platform, projectile, and enemy drone is placed by constructing a compound model matrix M:

```
M = T(position) · R_y(yaw) · R_x(pitch) · S(scale_x, scale_y, scale_z)
```

In the first-person viewmodel pass, the weapon hierarchy extends this transform relative to the camera base coordinate frame: `M_gun = T(camPos + offset) · R_cam · T(localPart) · S(partScale)`, where offset incorporates walking bobbing and recoil.

9.2 View and projection matrices

The view matrix V transforms world-space coordinates into the synthetic camera's coordinate frame via an orthonormal basis (R, U, -F):

```
V = glm::lookAt(camPos, camPos + forward, vec3(0, 1, 0))
```

The perspective projection matrix P simulates optical perspective by scaling distant coordinates inversely proportional to depth (1/z): `P = glm::perspective(radians(75.0f), aspect, 0.08f, 180.0f)`. For the 2D HUD overlay, parallel orthographic projection is used: `P_ortho = glm::ortho(0.0f, width, height, 0.0f, -1.0f, 1.0f)`.

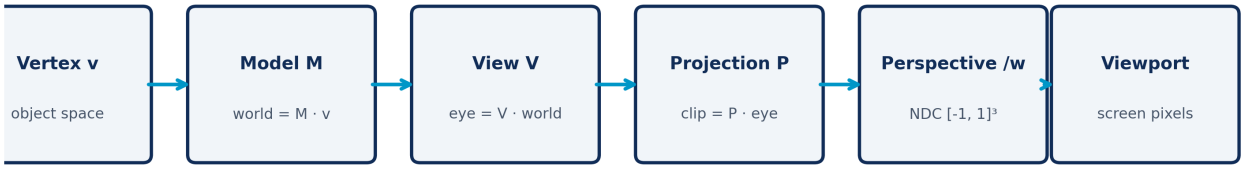


Figure 9.2 Complete 3D transformation and projection coordinate pipeline implemented in Chrono-Dash.

9.3 Surface Normal Transformation

When an object undergoes non-uniform scaling (e.g., platforms scaled 26m wide by 1.2m high), multiplying surface normals by the standard model matrix distorts perpendicularity. To maintain orthogonality, the vertex shader transforms normals using the transpose of the inverse of the upper-left 3x3 model matrix: `Normal = mat3(transpose(inverse(uModel))) * aNormal`.

10. CG Concepts II: Lighting, Shading & Materials

Per-fragment Blinn-Phong illumination, dynamic point-light attenuation, and emissive neon materials.

10.1 The comprehensive lighting model

The fragment shader evaluates per-pixel lighting from five distinct terms:

1. **Ambient:** Cool baseline ambient tint: `I_amb = vec3(0.09, 0.10, 0.15) * uColor`.

2. **Directional Twilight Sun:** Overhead light with Lambertian diffuse: `max(N · L, 0.0) * C_sun * uColor`.

3. **Blinn-Phong Specular:** Uses halfway vector `H = normalize(L + V)` to compute specular highlights: `pow(max(N · H, 0.0), shininess) * C_spec`.

4. **Dynamic Antigravity Core Point Light:** Omnidirectional point illumination at (0, 10.5, 0) decaying via distance attenuation: `Att(d) = 1.0 / (1.0 + 0.04*d + 0.006*d²)`.

5. **Muzzle Flash Point Light & Rim Glow:** Brief bright cyan burst at weapon tip on firing (`Att = 1 / (1 + 0.25d + 0.12d²)`) and Fresnel rim silhouette: `(1 - N · V)^3.2`.

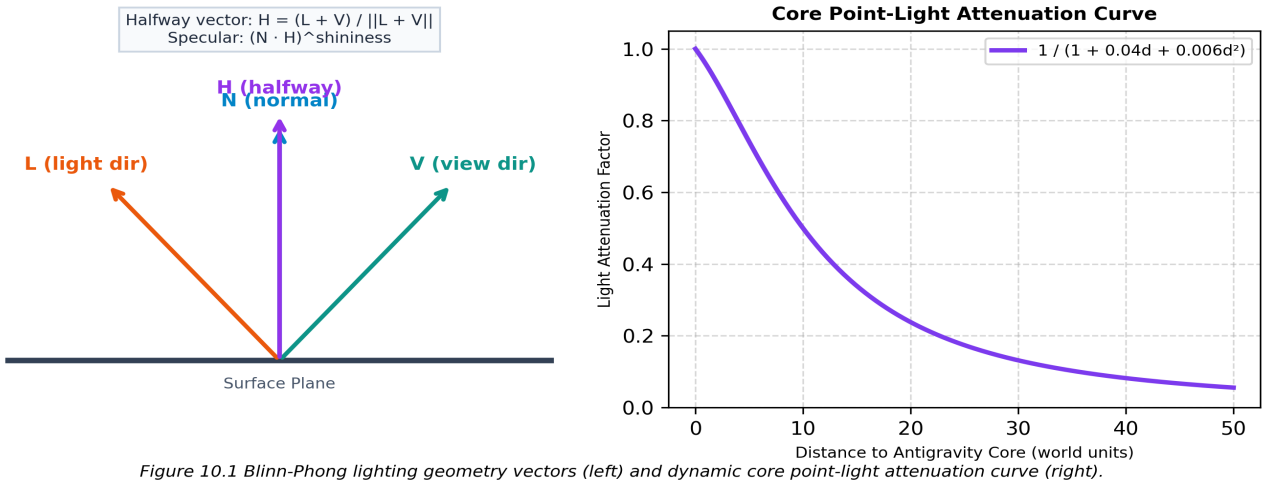


Figure 10.1 Blinn-Phong lighting geometry vectors (left) and dynamic core point-light attenuation curve (right).

10.2 Material parameter specifications

MATERIAL TYPE	BASE COLOR (Kd)	EMISSIVE COLOR	SHININESS
Dark Carbon Platform	vec3(0.05, 0.06, 0.09)	vec3(0.0, 0.0, 0.0)	32.0 (Metallic)
Neon Cyan Trim Strips	vec3(0.10, 0.85, 1.00)	vec3(0.15, 1.28, 1.50)	64.0 (Glow)
Neon Amber North Trim	vec3(1.00, 0.40, 0.08)	vec3(1.50, 0.60, 0.12)	64.0 (Glow)
Neon Purple High Trim	vec3(0.80, 0.20, 1.00)	vec3(1.20, 0.30, 1.50)	64.0 (Glow)
Weapon Shroud (White/Orange)	vec3(0.92, 0.94, 0.96)	vec3(0.0, 0.0, 0.0)	48.0 (Sleek)
Drone Sensor Eye (Red)	vec3(1.00, 0.10, 0.15)	vec3(2.50, 0.20, 0.30)	96.0 (Intense)
Laser Projectiles	vec3(0.15, 0.90, 1.00)	vec3(0.45, 2.70, 3.00)	128.0 (Laser)

11. CG Concepts III: Dynamic Viewmodel Animation & Physics

Procedural harmonic motion, mouse-look lag sway, and kickback recoil physics bring the weapon to life.

11.1 First-person viewmodel dynamics

Rather than rendering a static 3D gun mesh, Chrono-Dash computes real-time procedural animations based on player movement and firing states:

• **Walk Bobbing Harmonics:** While grounded and moving, the weapon oscillates along horizontal and vertical sine waves: `bobX = sin(t * 8.0) * 0.008` and `bobY = |cos(t * 8.0)| * 0.007`, creating realistic footstep cadence.

• **Mouse Look Lag Sway:** Cursor delta motion creates rotational and translational inertia lag (`gunYawSway -= dx * 0.003`, `gunPitchSway += dy * 0.003`), damped smoothly each frame by `0.85^(dt*60)`.

• **Weapon Kickback Recoil:** Discharging a laser bolt kicks the gun back along local Z (`-0.08m`) and pitches the barrel upward by 6.0°. Recoil decays linearly back to zero over 125ms via `recoil = max(0, recoil - dt * 8.0)`.

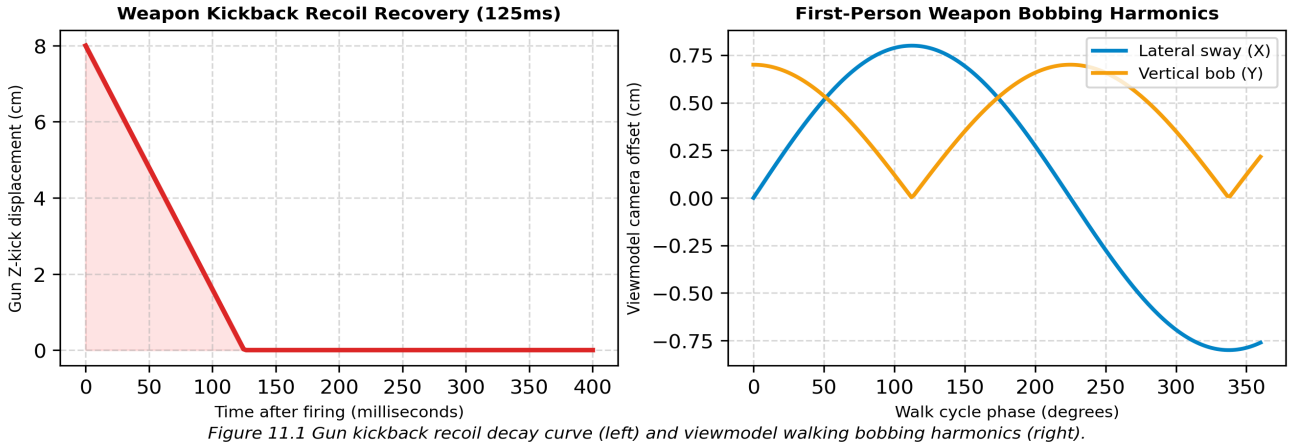


Figure 11.1 Gun kickback recoil decay curve (left) and viewmodel walking bobbing harmonics (right).

11.2 Particle physics and destruction FX

Upon destroying an enemy drone, the engine spawns 30 spark and debris particles with random 3D velocities: `vel = vec3(rand(-6,6), rand(-4,8), rand(-6,6))`. Particles travel under ballistic gravity, shrinking and fading over lifetimes of 0.4s to 0.85s. Inside the zero-G field, ambient floating motes drift smoothly around the player.

12. CG Concepts IV: Depth, Fog & Rendering Passes

Hardware Z-buffering, depth buffer clearing for weapon isolation, and atmospheric distance fog.

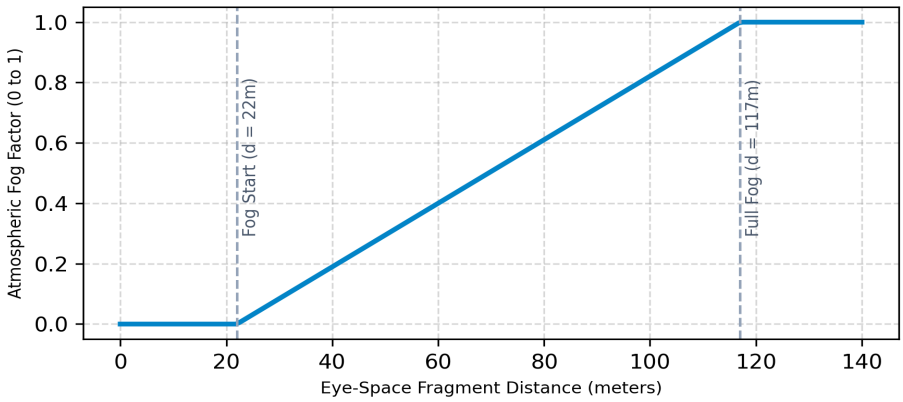
12.1 Depth buffer & hidden surface removal

Hardware depth testing (GL_DEPTH_TEST with glDepthFunc(GL_LESS)) is enabled to resolve polygon occlusion. The 24-bit Z-buffer stores fragment depth values. Near plane (0.08m) and far plane (180.0m) are tuned to maximize depth precision in the combat zone while keeping z-fighting negligible.

12.2 Atmospheric distance fog

The fragment shader measures fragment eye-space distance `d = length(FragPos - uViewPos)` and calculates an atmospheric fog blending factor: `fog = clamp((d - 22.0) / 95.0, 0.0, 1.0)`. Geometry closer than 22m remains pristine, while distant platforms smoothly blend into the deep twilight sky color (0.025, 0.035, 0.075), establishing strong spatial depth cues.

Figure 12.1 Atmospheric distance fog blending factor: clamp((d - 22.0) / 95.0, 0, 1)



12.3 The three-pass rendering pipeline

1. **3D World Pass:** Draws platforms, floor grid, Antigravity Core, debris, drones, laser bolts, and particles with depth test enabled.
2. **Viewmodel Weapon Pass:** Clears ONLY the depth buffer: `glClear(GL_DEPTH_BUFFER_BIT)`. The gun is drawn in camera-relative view space. Resetting the depth buffer **guarantees the gun never clips into world geometry**.
3. **2D HUD Pass:** Depth testing is disabled (`glDisable(GL_DEPTH_TEST)`). An orthographic pixel-space matrix draws the score, streak gauge, health/ammo meters, and crosshair reticle on top.

13. Physics, Collision & Antigravity Mechanics

Analytic geometric intersection tests and dual-zone gravitational integration without external physics libraries.

13.1 Numerical integration and gravity states

Entity movement is integrated each frame via semi-implicit Euler integration: `v(t+dt) = v(t) + a·dt`, followed by `p(t+dt) = p(t) + v·dt`. Frame delta time is clamped to max 50ms to prevent tunneling through thin platforms during performance spikes. Proximity to the core (d < 16.5m) dynamically transitions the downward acceleration from g = -25.0 m/s² to g = -2.2 m/s².

13.2 Collision detection primitives

COLLISION TEST	GEOMETRIC FORMULA	GAMEPLAY APPLICATION
Player vs Platform Top	foot_old >= max.y - 0.1 && foot_new <= max.y (with X/Z overlap)	Snaps player feet to platform surface, resets vy = 0, sets isGrounded = true
Player vs Platform AABB	boxA.intersects(boxB) across [min, max] on X, Y, Z	Prevents player from walking through elevated platform walls
Laser vs Drone / Player	length(p.pos - target.pos) < (p.radius + target.radius)	Detects laser hits (cyan on drones, purple on player), deals damage, spawns sparks
Laser vs Platform AABB	platformAABB.contains(p.pos)	Terminates laser projectile upon impacting solid metallic platforms
Void Boundary Detection	player.pos.y < -12.0m	Triggers safety repulsion blast (+19.0 m/s), inflicts 12 HP penalty

13.3 Drone collision & evasive response

Hostile drones possess a 1.8m bounding volume. When hit by a player projectile, the drone takes 30 damage points, triggers a 0.15s white hit-flash, and produces a directional spark burst. Drones with health <= 0 are destroyed, awarding 100 points multiplied by the current streak multiplier and triggering an omnidirectional particle explosion.

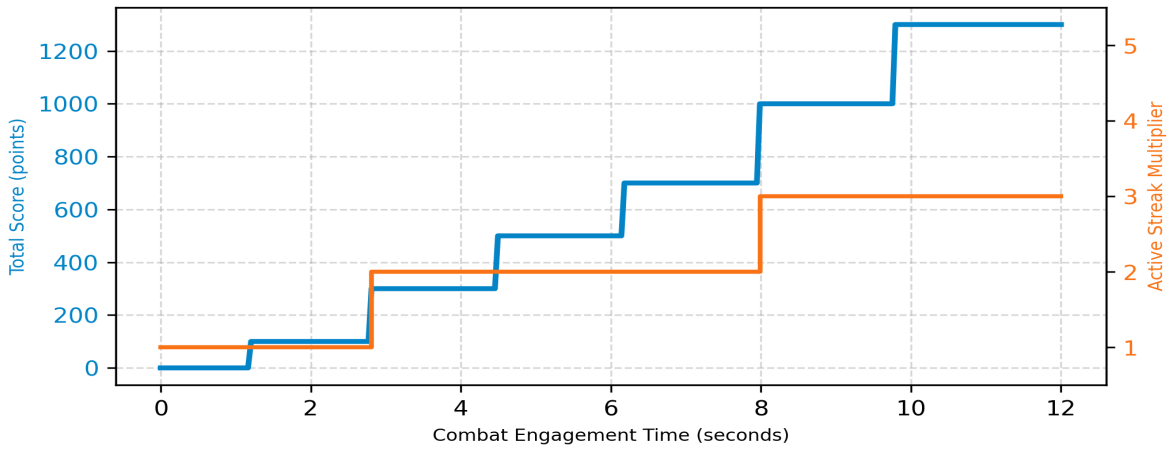
14. Scoring, Progression & Level Walkthrough

High-skill multiplier mechanics, active countdown timers, and combat pacing.

14.1 Score and streak mechanics

Chrono-Dash rewards aggression, precision, and continuous engagement. Eliminating a drone awards 100 base score multiplied by the active streak multiplier: `Score += 100 * streakMultiplier`. Each confirmed kill resets the streak countdown timer to 4.5 seconds. Consecutive kills advance the multiplier tiers: 1-2 kills = 1x, 3-4 kills = 2x, 5-8 kills = 3x, and 9+ kills = 5x max multiplier. If the 4.5-second timer expires before another kill is scored, the streak counter resets to zero and the multiplier reverts to 1x.

Figure 14.1 Score accumulation with consecutive kill streak multipliers (1x -> 2x -> 3x -> 5x)



14.2 Tactical combat walkthrough

COMBAT PHASE	TACTICAL OBJECTIVE & ACTION	GRAPHICS PRINCIPLE DEMONSTRATED
Phase 1: Ground Skirmish	Engage perimeter drones from the main platform; sprint and jump	Kinematic ground physics, AABB landing, weapon sway
Phase 2: Zero-G Incursion	Leap into the central Antigravity Core; ascend with Spacebar	Antigravity buoyancy, point-light illumination, floaty drag
Phase 3: Aerial Dogfight	Dodge purple drone lasers mid-air; lead targets with cyan lasers	Dynamic point light attenuation, sphere-sphere hit detection
Phase 4: High Perch Vantage	Thrust onto elevated West/East platforms (y=11.5m) to reload	Platform boundary resolution, tactical reloads, 5x score streak

15. Interface & HUD

A pure vector stroke typography engine providing real-time combat telemetry without external font files.

15.1 HUD telemetry elements

ELEMENT	SCREEN LOCATION	TELEMETRY & VISUAL PRESENTATION
Score Display	Top-Right Panel	Large white vector text: 'SCORE: 12500' with cyan corner bracket accents
Streak Multiplier	Top-Right Panel	Orange text: 'STREAK: 5X' with depleting horizontal countdown gauge bar
Health Status	Bottom-Left Panel	'CHRONO_HUNTER', 'HP: 100' with segmented cyan health bar (turns red < 35%)
Ammo Telemetry	Bottom-Right Panel	'PULSE_CANNON', 'AMMO: 30 / 90' with orange ammo gauge and 'RELOADING...' notice
Antigravity Badge	Top-Center Pill	Glowing badge: '[ANTIGRAV: ACTIVE (ZERO-G)]' (purple) vs '[GRAVITY: 1.0G NOMINAL]'
Crosshair Reticle	Screen Center	Center cyan aiming dot with 4 dynamic corner brackets that expand with recoil
Damage Vignette	Screen Perimeter	Red translucent border flash (alpha decays over 0.5s) upon taking laser or void hits
Mission Failed Modal	Screen Center	Red bordered dialog: 'MISSION FAILED', final score display, 'PRESS [R] TO RESPAWN'

15.2 Procedural vector stroke rasterization

To avoid external font files (.ttf) or texture atlas dependencies, the UIRenderer class implements an analytic vector stroke engine. Characters (0-9, A-Z, punctuation) are stored as normalized 1D line segments between points in a [0, 1] square. For each stroke from point A to B, the engine computes the orthogonal normal n = (-dy, dx) / length * (thickness / 2) and generates two triangles forming a screen-space quad {A+n, A-n, B+n, B-n}. This produces ultra-crisp, resolution-independent sci-fi digital typography that renders at any window resolution without pixelation.

16. Testing, Validation & Known Limitations

Verification harnesses, validation checklists, and known architectural boundary conditions.

16.1 Automated build and smoke verification

The project provides a dual-path automated build harness (build.bat):

• **CMake Pipeline:** Invokes CMake 3.16+ with Ninja generator, compiling all source modules and linking OpenGL/GLFW libraries into ChronoDash.exe in ~4.8 seconds.

• **Direct MinGW Pipeline:** Fallback command compiling all source translation units directly via g++ -std=c++17 -O2 with zero warnings.

16.2 Recommended interactive evaluation checklist

1. Launch ChronoDash.exe; confirm 4x MSAA rendering, deep twilight sky, and crisp wireframe floor grid.

2. Test WASD movement and mouse freelook; observe procedural weapon walking bobbing and mouse-lag sway.

3. Press Spacebar on the main platform to confirm standard 1.0G parabolic jumping (peak height ~1.7m).

4. Step into the central sphere radius; confirm HUD badge displays '[ANTIGRAV: ACTIVE (ZERO-G)]'.

5. Hold Spacebar inside the core to verify continuous 3D vertical climbing; hold Shift/Ctrl to verify active descent.

6. Discharge Pulse Cannon (Left Mouse Button); observe weapon kickback recoil, barrel flash, and cyan laser trajectory.

7. Engage hostile drones; confirm red hit flashes, soundless particle explosions, and streak multiplier progression (1x -> 2x -> 3x -> 5x).

8. Fall off platform below y = -12.0m; verify safety repulsion boost (+19.0 m/s) and red damage vignette.

16.3 Known limitations

• **Audio Subsystem:** Visual and lighting cues (muzzle flash, explosions, damage vignette) are fully implemented; audio sound effects are currently visual-only.

• **Shadow Mapping:** Lighting uses direct Blinn-Phong and distance attenuation; shadow volumes / depth texture shadow mapping are planned for future iterations.

• **Mesh Simplicity:** Meshes are procedurally generated cubes, spheres, and octahedrons; skeletal rigging or imported OBJ/FBX models are not used to keep the project 100% self-contained.

17. Conclusion, Future Scope & References

Chrono-Dash demonstrates how foundational computer graphics algorithms combine to create a fast, cohesive 3D game.

17.1 Conclusion

Chrono-Dash: Antigravity Arena demonstrates that complex commercial game engines are not strictly necessary to build engaging, responsive 3D games. By implementing affine matrix transformation chains, look-at camera orthonormal bases, perspective projection, GLSL programmable shaders, Blinn-Phong illumination, point-light attenuation, procedural geometry, AABB collision, and a 3-pass rendering architecture, the project establishes a comprehensive, mathematically rigorous showcase of modern OpenGL 3.3 computer graphics principles.

17.2 Future scope

• Implement directional cascaded shadow maps (CSM) and omnidirectional point-light shadow cubemaps for the Antigravity Core.

• Integrate spatial 3D positional audio using OpenAL or FMOD Core with Doppler shift effects in the zero-G field.

• Add GPU compute shader particle simulation to render thousands of interactive zero-G plasma motes and debris.

• Implement procedural level generation allowing dynamic floating arena configurations and multiple mission sectors.

17.3 References and project resources

1. Chrono-Dash source repository: CgGameCie3/ (include/, src/, CMakeLists.txt, build.bat, run.bat).

2. J. F. Blinn, 'Models of Light Reflection for Computer Synthesized Pictures', ACM SIGGRAPH 1977 - Blinn-Phong reflection formulation.

3. E. Angel and D. Shreiner, 'Interactive Computer Graphics: A Top-Down Approach with WebGL/OpenGL', Pearson Education.

4. Joey de Vries, 'LearnOpenGL: Creating a 3D Game with C++ and OpenGL', learnopengl.com.

5. GLFW 3.3 and GLAD 2.0 API Documentation: glfw.org and glad.sh.

6. OpenGL Mathematics (GLM) Library Documentation: glm.g-truc.net.

7. Academic Institutional Identity: Army Institute of Technology, Pune (Department of Computer Engineering, motto 'Onward to Glory').